

AI時代のFlutter開発スペシャル by クラスメソッド

【Flutter × AI】

AI は Material Design 3 を使って Widget を実装できるのか

～Figmaデータの整備度がAIの出力に与える影響～

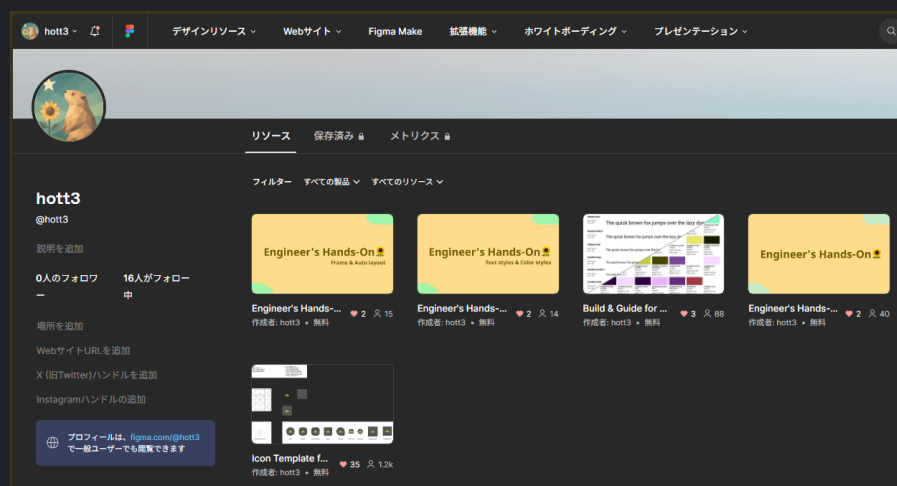
2026.5.11

 hott3

X @hott3_

自己紹介

- 堀田 誠 (X : @hott3_)
- モバイルエンジニア (Flutter)
- ドコドア株式会社
- 関心：
 - モバイル、デザイン、人が作るもの、ネコ 🐱



TL;DR 🌿

- 最近のAIモデルはMD3を理解している
- デザインデータが整っていなくても、ほぼ見た目通りのコードを出力することができる
- エンジニアは、AIに何を期待するかを理解し、戦略を立てる必要がある
 - どんな情報がAIの出力に影響を与えるのか
 - その情報を整えるためにどこまで工数をかけるべきか

こんなこと聞いたことありますか？

- 💬 「AIにFigmaを渡せば、もう一瞬でコードが出るよね」
- 💬 「いや、まずはデザインシステムを整備しないと、AIも迷っちゃうよ」
- 💬 「結局、レイヤーをちゃんと構造化（Auto Layoutなど）してないと使い物にならないよ」
- 💬 「これからはAIへの指示書として『DESIGN.md』を同梱するのが当たり前になりそう」

🤔 「どこまでデザインデータを整備するか？」

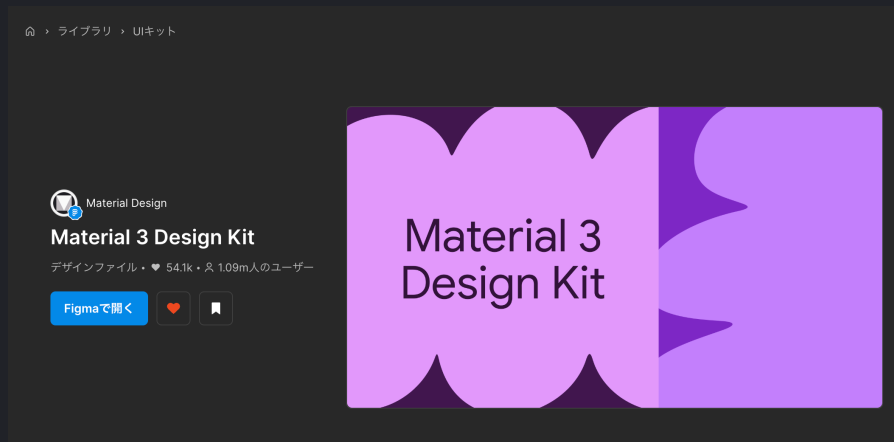
- AIが出力したコードに対して、人間の手直しが発生する可能性
- デザインデータを整えるための工数が確保できない可能性
- デザインデータの構造が、出力されるFlutterのコードの品質に与える影響が不明確

→ どこまでデザインデータを整備するべきか、明確な指針がない

検証

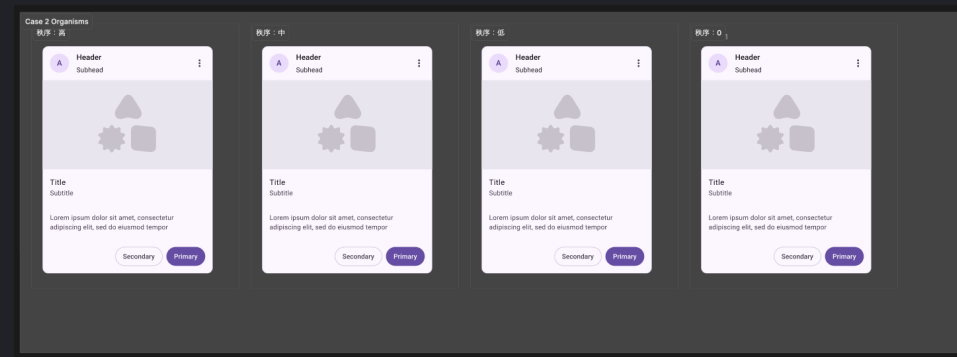
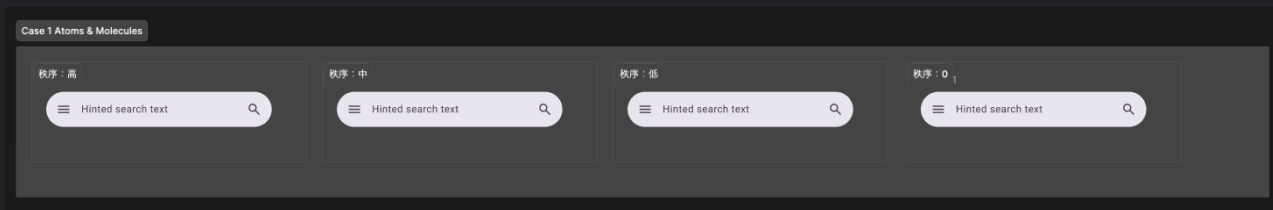
Figma の UI Kit を使ったデザインデータで Material Design 3 のコンテキストを渡せば、AIはMD3準拠のWidgetを選定しやすくなるのではないか？

→ Widgetが選定されやすい条件と、そうでない条件を発見することで、**どこまでデザインデータを整えるべきかの指針**を探る



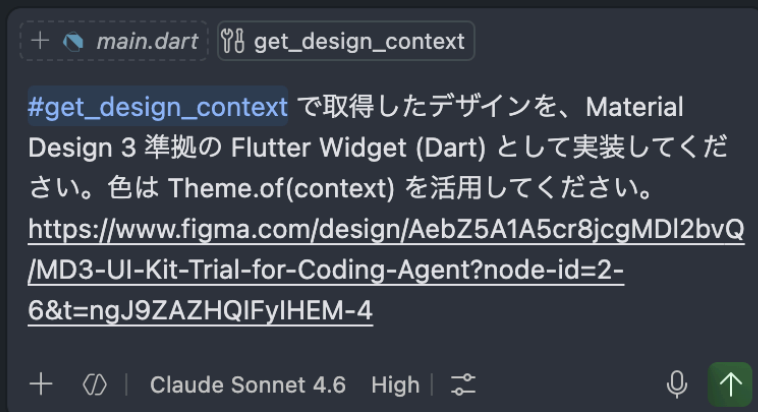
検証手順

1. FigmaのUI Kitを使って、2つのケースのデザインデータを用意
2. 見た目を維持しつつ、デザインデータを4つの構造レベルに変更する
3. AIに同一プロンプトを与え、それぞれのデザインのFlutterコードを出力させる
4. 出力されたコードを、MD3準拠のWidget選定、見た目の再現度の観点で評価する



検証で固定した条件

- Flutter: 3.41.6 / Dart: 3.11.4
- VS Code GitHub Copilot Agent mode を使用
- **Claude 4.6 Sonnet** を使用
- Figma Remote Server を利用 `get_design_context` でデザイン情報を取得
 - `get_screenshot` は指定しない
- Skillsの影響を除外するため Agent Skillsは未使用
- 同一プロンプトを使用



```
+ main.dart get_design_context
```

#get_design_context で取得したデザインを、Material Design 3 準拠の Flutter Widget (Dart) として実装してください。色は Theme.of(context) を活用してください。

<https://www.figma.com/design/AebZ5A1A5cr8jcgMDI2bvQ/MD3-UI-Kit-Trial-for-Coding-Agent?node-id=2-6&t=ngJ9ZAZHQIFyIHEM-4>

+ | Claude Sonnet 4.6 High | 上

ローカル 既定の承認

検証で変更した条件

Figmaのデザインデータの整備度を4段階で変更

構造レベル	コンポーネント機能	レイヤー名	スタイル/バリエーション機能	オートレイアウト機能	レイヤー構造	見た目
高	維持	維持	維持	維持	維持	維持
中	×	×	維持	維持	維持	維持
低	×	×	×	×	維持	維持
0	×	×	×	×	×	維持

→ この構造レベルの変化をもって、デザインデータがAIの出力に与える影響を検証する

構造レベル：高
UI Kitそのままのデザイン

構造レベル	コンポーネント機能	レイヤー名	スタイル/バリアブル機能	オートレイアウト機能	レイヤー構造	見た目
高	維持	維持	維持	維持	維持	維持



構造レベル：中

Case 1: 検索バー

コンポーネントを切り離しレイヤー名を変更したデザイン

構造レベル	コンポーネント機能	レイヤー名	スタイル/バリアブル機能	オートレイアウト機能	レイヤー構造	見た目
中	✗	✗	維持	維持	維持	維持

□ 秩序：中

□□ Frame 1

□□ Frame 2

□□ Frame 3

☰ Frame 4

□□ Frame 5

⌘ Frame 6

≡ Icon 1

□□ Frame 7

T Text 1

□□□ Frame 8

□□ Frame 9

☰ Frame 10

□□ Frame 11

秩序：中

</>

Frame 1

☰ Hinted search text

🔍

576 × 260

© 2026 hott3

11

構造レベル：低

Case 1: 検索バー

デザイントークンとオートレイアウトを削除したデザイン

構造レベル	コンポーネント機能	レイヤー名	スタイル/バリアブル機能	オートレイアウト機能	レイヤー構造	見た目
低	×	×	×	×	維持	維持

□ 秩序：低

Frame 1

Frame 2

Frame 3

Frame 4

Frame 5

Frame 6

Frame 7

Icon 1

Frame 8

Frame 9

Frame 10

Frame 11

Icon 2

Frame 12

Text 1

Frame 13

Frame 14

Frame 15

Frame 16

秩序：低 </>

Frame 1

≡

Hinted search text

🔍

797 × 276

© 2026 hott3

構造レベル：0
見た目が保てる最低限のレイヤー構造を残したデザイン

構造レベル	コンポーネント機能	レイヤー名	スタイル/バリアブル機能	オートレイアウト機能	レイヤー構造	見た目
0	×	×	×	×	×	維持

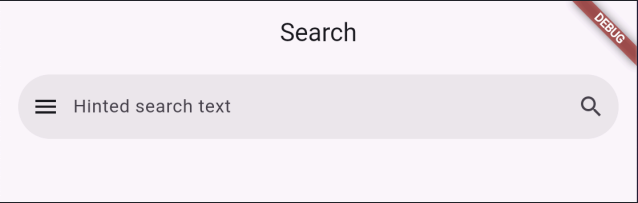
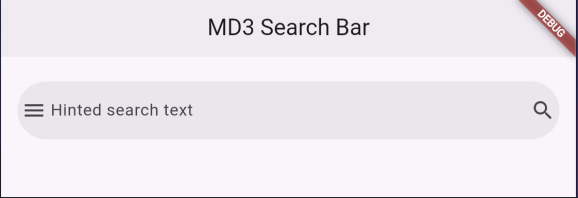
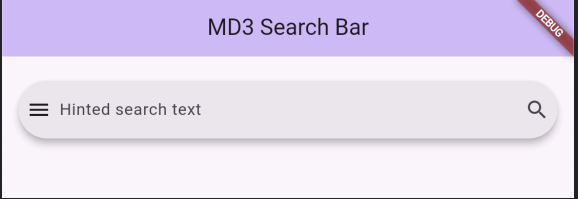


結果：構造レベル0 のデザインデータをAIに渡して出力されたコード

見た目しか維持されていないデザインデータでも、MD3準拠のWidget選定はされている

```
SearchBar(  
  hintText: 'Hinted search text',  
  hintStyle: WidgetStatePropertyAll(  
    Theme.of(context,).textTheme.bodyLarge?.copyWith(color: colorScheme.onSurfaceVariant),  
  ),  
  textStyle: WidgetStatePropertyAll(  
    Theme.of(context,).textTheme.bodyLarge?.copyWith(color: colorScheme.onSurface),  
  ),  
  backgroundColor: WidgetStatePropertyAll(colorScheme.surfaceContainerHigh),  
  shadowColor: const WidgetStatePropertyAll(Colors.transparent),  
  leading: Icon(Icons.menu, color: colorScheme.onSurface),  
  trailing: [Icon(Icons.search, color: colorScheme.onSurfaceVariant)],  
);
```

MD3準拠のWidget選定、見た目の再現度

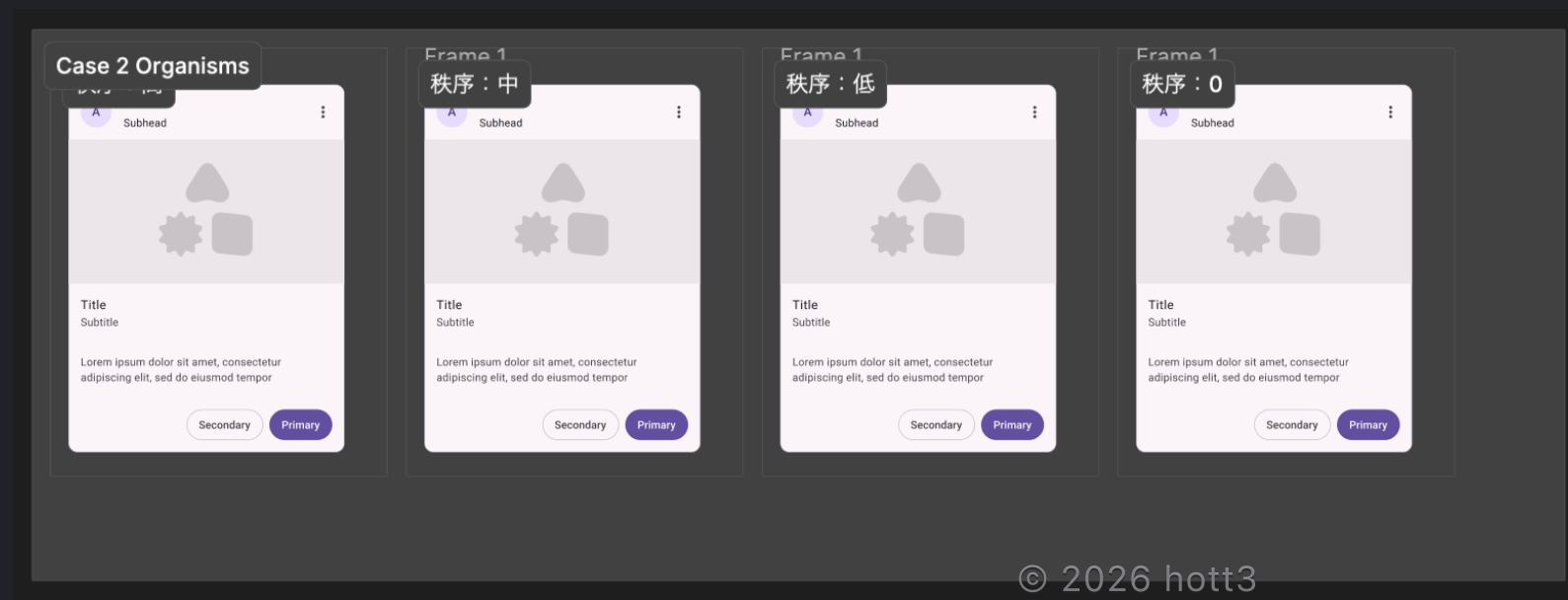
構造レベル	MD3準拠のWidget選定	見た目の再現度
高	SearchBar が使用されている ✓	
中	SearchBar が使用されている ✓	
低	SearchBar が使用されている ✓	
0	SearchBar が使用されている ✓	

→ MD3準拠のWidgetが選定され、見た目通りのコードが出力されている

Case 2: 商品カード

Case 2: 商品カード

- 構造レベル：高
 - UI Kitそのままのデザイン
- 構造レベル：中
 - コンポーネントを切り離しレイヤー名を変更したデザイン
- 構造レベル：低
 - デザイントークンとオートレイアウトを削除したデザイン
- 構造レベル：0
 - 見た目が保てる最低限のレイヤー構造を残したデザイン



構造レベル：0 のデザインデータをAIに渡して出力されたコード

見た目しか維持されていないデザインデータでも、MD3準拠のWidget選定はされている

～略～

```
@override
Widget build(BuildContext context) {
  final colorScheme = Theme.of(context).colorScheme;
  final textTheme = Theme.of(context).textTheme;

  return Card(
    clipBehavior: Clip.antiAlias,
    child: SizedBox(
      width: 360,
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          _Header(
            ～略～
          ),
          _ImageArea(imageWidget: imageWidget, colorScheme: colorScheme),
          _Content(
            ～略～
          ),
        ],
      ),
    ),
  );
}
```

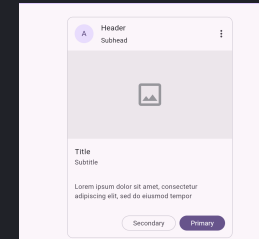
構造レベル

MD3準拠のWidget選定

見た目の再現度

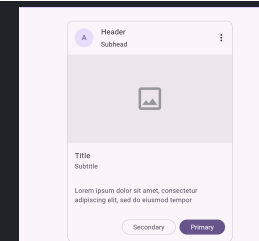
高

Card / OutlinedButton / FilledButton が使用されている ✓



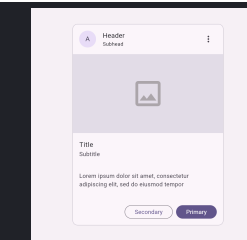
中

Card / OutlinedButton / FilledButton が使用されている ✓



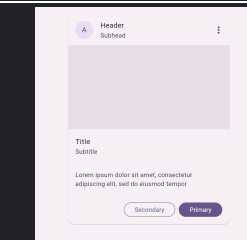
低

Card / OutlinedButton / FilledButton が使用されている ✓



0

Card / OutlinedButton / FilledButton が使用されている ✓



→ MD3準拠のWidgetが選定され、見た目通りのコードが出力されている

検証結果

- 構造レベルを下げても、MD3準拠のWidget選定はされ、見た目通りのコードが出力されている
 - つまり、AIは「これがSearchBarだ」と推論できている

→ MD3のUI Kitを使うことが、AIに対して「このデザインはMD3のコンポーネントだ」という強い文脈を与られると予想していたが、デザインデータを破壊してもWidgetを選定できていた

検証中に気づいたこと

🤔なぜ構造レベル0のデザインデータでも、MD3準拠のWidget選定がされていたのか？

- Case 1の構造レベル低でスクリーンショットを取得していた
 - Figma MCP Serverの `get_design_context` のみ指定し、デザインデータを利用する想定だった

#get_design_context で取得したデザインを、Material Design 3 準拠の Flutter Widget (Dart) として実装してください。色は Theme.of(context) を活用してください。
<https://www.figma.com/design/AebZ5A1A5cr8jcgMDI2bvQ/MD3-UI-Kit-Trial-for-Coding-Agent?node-id=2-9&t=ngJ9ZAZHQIFyIHEM-4>

`get_design_context` `main.dart:54-90`

Searched for tool usage in the project

Get Design Context を実行しました – figma (MCP サーバー)

Considered calling get_design_context on individual nodes

Get Design Context を実行しました – figma (MCP サーバー)

`file.png` 🔄

Analyzed design for implementing MD3 Search Bar in Flutter

デザインが取得できました。MD3 の SearchBar です。実装前に既存ファイルとバージョンを確認します。

○

→ つまり、AIがスクリーンショットも取得して視覚情報をもとに見た目の実装を行っていた

新しい仮説とそこからわかること

事実として、AIはレイヤー構造が崩れていても、視覚情報からUIデザインを推論している（Flutter と MD3 という世界中で広く公開されているパターンで学習されているために、AIが必要なWidgetを推論しやすかった可能性がある）

→ つまり、
構造化されていないデザインデータがAIの出力の品質を、必ず下げるわけではない

これからの行動指針 「なぜ注力するかで使い分ける」

✗ デザインデータを整えることにこだわる

- Auto Layout を完璧に整えることにこだわる
- レイヤー名をすべて正式名称に統一することを必須とする
- すべてのデザイン知識をAIに与えられると思い込む

✓ 目的に応じた必要なデザインから戦略的に整える

- 公開パターン（MD3など）
 - AIは学習済みなので、完璧な構造化は不要
 - デザインデータを用意するか目的に応じて取捨選択し、余分な中間成果物の作成に工数をかけない
- 独自ブランドデザイン
 - 戦略的にAIに情報を与える必要がある
 - DESIGN.md やデザイントークンで、必要な知識を優先度高く伝える
 - すべての完璧に整えるのではなく、「何を伝えるか」を選別する

TL;DR (Again) 🌻

- 最近のAIモデルはMD3を理解している
- デザインデータが整っていなくても、ほぼ見た目通りのコードを出力することができる
- エンジニアは、AIに何を期待するかを理解し、戦略を立てる必要がある
 - どんな情報がAIの出力に影響を与えるのか
 - その情報を整えるためにどこまで工数をかけるべきか

→ 中核的なブランドデザインの実装は、人間とAIがどちらも利用できるように設計し、戦略的に進めることが重要だと考えます！

参考資料

Material 3 Design Kit | Figma

<https://www.figma.com/community/file/1035203688168086460>

Figma MCP の get_design_context は何を返すのか — 実データで読み解く中間表現

<https://zenn.dev/yokkomystery/articles/932cacd7728188>

Set up the remote server (recommended) | Developer Docs

<https://developers.figma.com/docs/figma-mcp-server/remote-server-installation/>